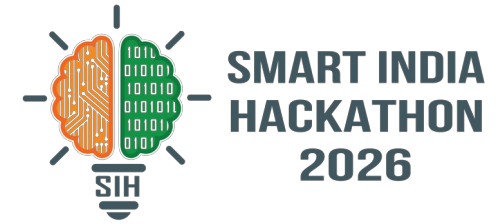


SMART INDIA HACKATHON 2026



TITLE PAGE

Problem Statement ID – SIH26159

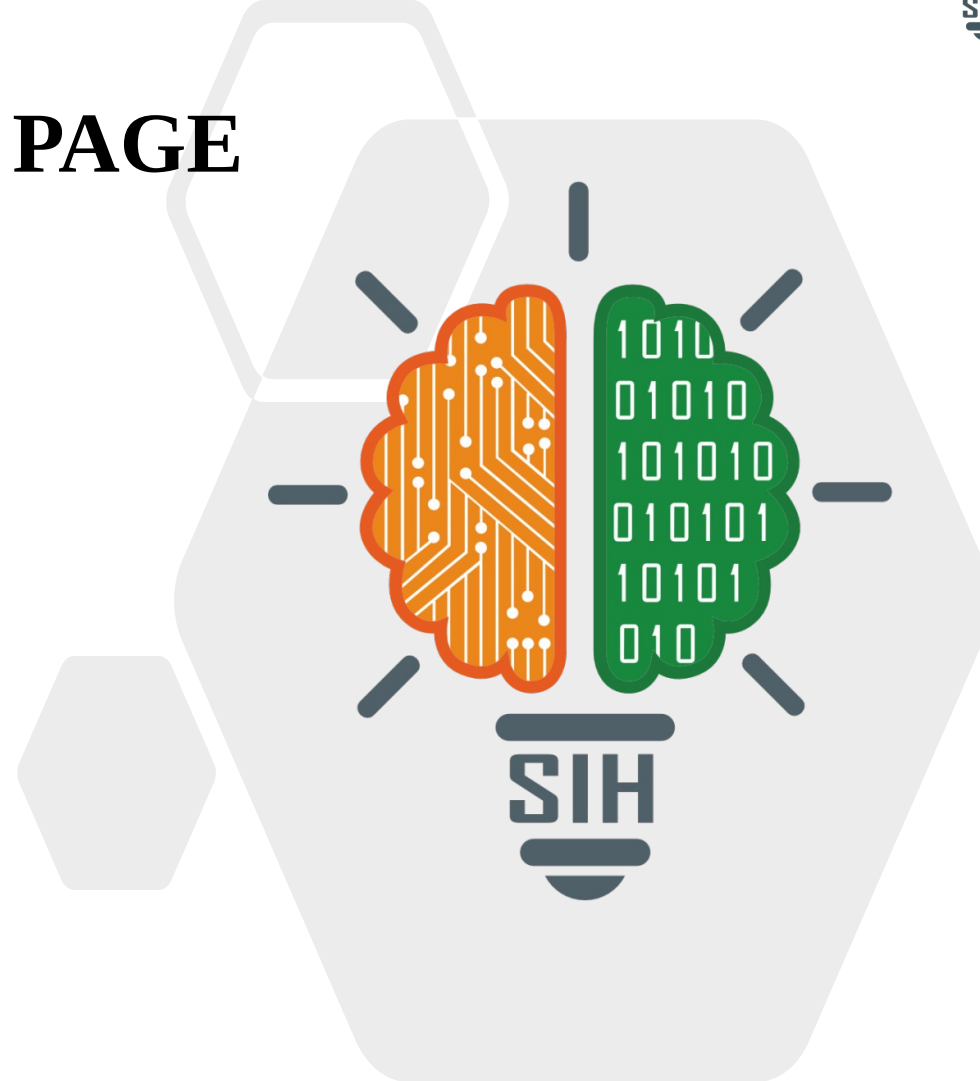
Problem Statement Title – [UNRESOLVED: problem statement title]

Theme – [UNRESOLVED: theme]

PS Category – Software

Team ID – [UNRESOLVED: team id]

Team Name (Registered on portal) – Team Zero-Day



IDEA TITLE

❖ SecureMailScope — passive cryptographic posture assessment for captured email traffic

The problem

Organisations know which mail servers they run. They cannot say which TLS versions those servers negotiated, which cipher suites they accepted, which sessions had no forward secrecy, or whether a STARTTLS upgrade was offered and then refused.

An active scanner cannot answer it either: it reports what a server does for a scanner today, not what it did for real clients during the period under investigation — and an authorised investigator often may not touch the host at all.

The solution

Read the packet captures the organisation already has. Report what the bytes show about how email was transported — and say so explicitly when the capture does not show something.

Passive and PCAP-first

The engine opens no socket. It never contacts a captured host, resolves a domain, scans anything, or sends capture contents anywhere — including to a language model. Proven by test, not asserted: socket constructors are replaced with functions that raise and the whole pipeline is run through them.

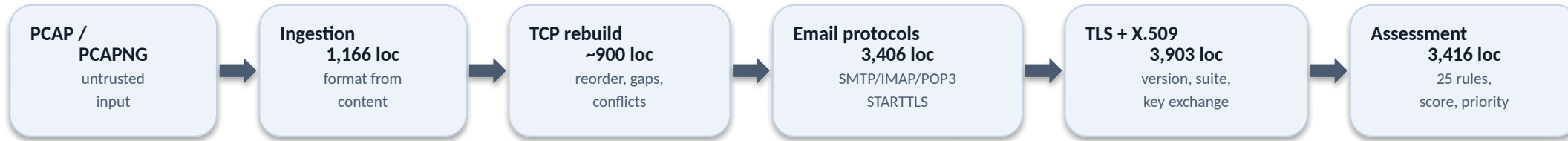
Evidence-backed, and honest about its limits

Every finding cites the packet numbers it was read from, the RFC it applies, a specific remediation, and its own limitation. Every value carries how it was obtained — OBSERVED, INFERRED, UNKNOWN or NOT_AVAILABLE — so a port-based guess can never be mistaken for a parsed dialogue.

A number that was not measured does not appear. A rule that could not be evaluated is excluded from both sides of the scoring fraction rather than counted as a pass or a violation.

TECHNICAL APPROACH

- Pipeline — every stage reads only what the previous one produced. Line counts are actual.



Cryptographic intelligence — 2,275 loc

fingerprints · server identity · drift between captures · correlation · evidence timeline · blast radius

Machine learning — 3,196 loc

93-feature schema · deterministic rarity baseline IN USE · Isolation Forest trained and held back · classifier NOT_VALIDATED

ONE canonical report model — 1,582 loc → JSON · HTML · PDF, parity-tested. The HTML report fetches nothing; the PDF has no URL resolver, so 'no external request' is structural.

Technologies

Engine Python 3.12 · Scapy 2.7 (dissection only) · Pydantic 2.9 · cryptography 50
 App FastAPI · SQLAlchemy · SQLite · Uvicorn
 ML scikit-learn 1.5 · NumPy 2.1
 Reports Jinja2 · ReportLab
 UI React 18 · TypeScript strict · Vite 8 · Tailwind · Recharts
 Test pytest · hypothesis · Playwright · Vitest · TShark (independent cross-check)

Local-first processing

Nothing in the diagram points outward. The only network path in the system is the browser talking to 127.0.0.1 — the application's own interface.
 The engine's dependency floor is three packages. A test walks the AST of every engine module to prove the web and ML stacks cannot reach the analysis path.
 scapy_guard replaces Scapy's neighbour resolver with one that raises — the one place the dissector could have spoken on the wire.

FEASIBILITY AND VIABILITY

- Feasible because it is built and measured. Every figure below is from the release commit, not an estimate.

Built and verified

1,354 Python tests pass; 1,364 with the TShark cross-check enabled
85 frontend tests · 5 browser end-to-end specs against the real backend, nothing mocked
mypy clean over 152 files · ruff clean
22-step browser-to-backend acceptance test, including stopping and restarting the backend and confirming the investigation survives
Demonstration rehearsed: 14 steps, 0 failures
Clean install from a bare checkout, engine-only and full

Measured performance

1,200 packets → 0.86 s, 270 MB peak
6,000 packets → 4.73 s, 625 MB peak
24,000 packets → 21.4 s, 1,800 MB peak
Ground truth matched exactly at every size
Thresholds were committed BEFORE the results they judge — the order is in the repository history
Limits: one machine, 16 logical CPUs, synthetic captures. A repeat run on a loaded machine measured 257 packets/s and FAILED the 500 packets/s threshold; that result is published, not discarded.

Security controls

Loopback binding is not a security model, so: host allowlist (DNS rebinding), explicit CORS origins with credentials off, per-installation token outside the repository at mode 0600
Bounded collection responses · magic-byte upload validation while streaming · error bodies with no path or stack trace
0 known vulnerabilities: pip-audit over 49 locked packages, npm audit over production and development trees
Hash-pinned lock verified with --require-hashes · CycloneDX SBOM for both trees
NOT a penetration test and NOT a security certification. Not safe to expose to a network, and not claimed to be.

Challenges, risks, and what is genuinely unresolved — stated, not hidden

TLS 1.3 encrypts the Certificate message. PERMANENT. No passive tool can read it. Reported NOT_AVAILABLE with the reason, never blank and never guessed. TLS 1.2 certificates are read and verified in full: dates, chain, hostname, key size, signature algorithm.

Supervised risk classification is NOT_VALIDATED for real-world use. Implemented and measured on synthetic data (macro-F1 0.5624), which does not establish real-world accuracy. No independent representative validation has been obtained, the interface says so on screen, and the classifier does not drive any finding or score.

Handshake completion is not verifiable and revocation is never checked. A capture has no traffic keys, and an OCSP or CRL request would break the passive rule. Three constants are asserted false or zero in every report by the test suite.

Deployment constraints. Memory is the binding limit — about 76 KB peak resident per packet — and two analyses share one process, so large captures are not claimed to run on the 8 GB machine the thresholds assume. No run on that hardware has been taken; the requirement is recorded NOT_VERIFIED. Analysis cannot be cancelled, and a test asserts the absence so the gap cannot be mistaken for a broken control.

IMPACT AND BENEFITS

- For authorised email-security investigation. These are potential benefits: the tool is demonstrated on synthetic captures and has no deployments, no users and no measured real-world outcomes.

Assessment without touching the host

The capture is already collected. Nothing is connected to, scanned or resolved — so an investigator with no authority to touch a production mail server can still assess how it protected traffic.

A cryptographic inventory of what was actually used

Not what a configuration file says. Negotiated versions, cipher suites, key exchanges and certificate details per session, read from the bytes on the wire.

Findings that carry their own evidence

Every finding cites the packet numbers it was read from, the RFC it applies, and its own limitation. An analyst can open the same packet in Wireshark and check.

Prioritised, specific remediation

25 rules under a versioned policy produce a priority band and a named remediation with its expected security effect — so the answer to 'what do I fix first' comes with a reason.

Drift between captures, attributed

The same server observed twice. If the client offered the same suites and the server selected differently, the change is attributable to the server. If the offers differed, the result is INCONCLUSIVE and says so.

Investigation reports that stand alone

JSON, HTML and PDF from one model, so their facts cannot disagree. The HTML opens offline and fetches nothing, which matters for an artefact that may be handed to a reviewer.

No financial saving, adoption figure, deployment or real-world detection rate is claimed: none has been measured.

- Standards the rules cite and the parsers implement. Every rule in the engine names the clause it applies.

TLS and certificates

RFC 8446 — TLS 1.3
RFC 5246 — TLS 1.2
RFC 9325 — Recommendations for Secure Use of TLS/DTLS (2022)
RFC 8996 — Deprecating TLS 1.0 and TLS 1.1
RFC 7457 — Known Attacks on TLS and DTLS
RFC 4492 — ECC Cipher Suites for TLS
RFC 5280 — X.509 Certificate and CRL Profile
RFC 6125 — Verification of Application Service Identity
NIST SP 800-52 Rev. 2 — Guidelines for TLS Implementations
IANA TLS Parameters registry — suites, groups, signature schemes

Email transport

RFC 5321 — Simple Mail Transfer Protocol
RFC 3207 — SMTP Service Extension for Secure SMTP over TLS
RFC 9051 — IMAP version 4rev2
RFC 1939 — Post Office Protocol version 3
RFC 2595 — Using TLS with IMAP, POP3 and ACAP
RFC 2606 — Reserved Top Level DNS Names (.invalid, used by every synthetic fixture)

Libraries

Scapy · Pydantic · python-cryptography · FastAPI · SQLAlchemy · scikit-learn · ReportLab · React · Vite
TShark (Wireshark) — used as an independent dissector to cross-check our own parse

Project research and documentation — written alongside the code, in this repository

docs/evidence-model.md — the four evidence statuses and how provenance is tracked · docs/scoring-methodology.md — the formula, the bands, and why a multi-capture headline is the weakest capture rather than an average
docs/limitations.md — what passive analysis cannot do · docs/threat-model.md — risks from untrusted captures and how each is bounded · docs/security-audit.md — the local threat model, the controls, and what the audit did not cover
docs/ml-methodology.md and docs/ml-model-card.md — why a deterministic rarity baseline was selected over a trained Isolation Forest by measurement, and what the models are not fit for · docs/ml-evaluation.md — the metrics and the synthetic dataset they describe
docs/performance-benchmarks.md — measurements, thresholds committed beforehand, and a published failing run · docs/requirements-matrix.md — every requirement mapped to a module, a test and an honest status